08 DYNAMIC MEMORY ALLOCATION • THEORY

02 dynamic arrays and objects

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

02 — Dynamic Arrays (`new[]` / `delete[]`) and Dynamic Objects

1. Allocating and Freeing Dynamic Arrays

CPP

```cpp
int n = 5;
int* arr = new int[n]{10, 20, 30, 40, 50};
// use arr[0] .. arr[4]
delete[] arr; // MUST use delete[], NEVER scalar delete!
arr = nullptr;
```

Strict Rule: Matching `new[]` with `delete[]` (`[expr.delete]`)

- `new T` **must** be paired with `delete ptr;`.
- `new T[n]` **must** be paired with `delete[] ptr;`.
- Calling scalar `delete ptr;` on memory allocated with `new T[n]` is **Undefined Behavior** (among other failures, destructors for elements `1..n-1` will not be called, and heap metadata may be corrupted).

2. Dynamic Classroom Objects

When `person* list = new person[3];` is executed, the **default constructor** of `person` is called 3 times (once for each element). When `delete[] list;` is executed, the **destructor** `~person()` is called 3 times (in reverse index order from index 2 down to 0).